

SPRINT 7 | LUCAN VIEIRA

Nivell 1

- Exercici 1

Calculadora de l'index de massa corporal

- Escriu una funció que calculi l'IMC ingressat per l'usuari/ària, és a dir, qui ho executi haurà d'ingressar aquestes dades. Pots obtenir més informació del seu càlcul en:
-> [Índice de masa corporal IMC que es y como se calcula.](https://muysalud.com/salud/indice-de-masa-corporal-imc-que-es-y-como-se-calcula/)
- La funció ha de classificar el resultat en les seves respectives categories

```
def indi_imc():
    try:
        altura = float(input("Introduce tu altura en metros: "))
        peso = float(input("Introduce tu peso en kilogramos: "))

        if altura <= 0 or altura > 2.51:
            raise ValueError("La altura debe ser mayor que 0 metros y menor o igual a 2.51 metros.")

        if peso <= 0 or peso > 600:
            raise ValueError("El peso debe ser mayor que 0 kilogramos y menor o igual a 600 kilogramos.")

        IMC = round(peso / (altura ** 2), 2)

    except ValueError as ve:
        print("ATENCIÓN: Por favor, ingresa un valor numérico válido para la altura y el peso.")
        return None

    return IMC

def clasificacion_imc(IMC):
    if IMC < 18.5:
        x = 'Según el cálculo tienes bajo peso, pero existen otros métodos más eficaces para valorar el IMC.'
    elif 18.5 <= IMC < 24.9:
        x = 'Según el cálculo tienes un peso normal, pero existen otros métodos más eficaces para valorar el IMC.'
    elif 24.9 <= IMC < 29.9:
        x = 'Según el cálculo tienes sobrepeso, pero existen otros métodos más eficaces para valorar el IMC.'
    else:
        x = 'Según el cálculo tienes obesidad, pero existen otros métodos más eficaces para valorar el IMC.'

    return x

Y = indi_imc()
print(Y)
clasificacion_imc(Y)
```

```
... 23.15

... 'Según el cálculo tienes un peso normal, pero existen otros métodos más eficaces para valorar el IMC.'
```

Hice el código basado en el sitio:

<https://muysalud.com/salud/indice-de-masa-corporal-imc-que-es-y-como-se-calcula/>

básicamente el programa está hecho para pedir las medidas metros y kilogramos del usuario y aplicar la fórmula de IMC que es el peso en kilogramos dividido por la estatura en metros cuadrados;

o sea, pide la altura y peso como variables y después de eso aplicar a la fórmula $IMC = \text{round}(\text{peso} / (\text{altura} ** 2), 2)$; después de esto - con el resultado - aplicamos las lecturas posibles del IMC.

por si acaso los valores informados por el usuario no sean en metro o kilogramos el programa va a presentar un error:

```
except ValueError as ve:
    print("ATENCIÓN: Por favor, ingresa un valor numérico válido para la altura y el peso.")
    return None
```

de hecho tenemos dos posibilidades de errores:

```
if altura <= 0 or altura > 2.51:
    raise ValueError("La altura debe ser mayor que 0 metros y menor o igual a 2.51 metros.")

    if peso <= 0 or peso > 600:
        raise ValueError("El peso debe ser mayor que 0 kilogramos y menor o igual a 600 kilogramos.")
```

he creado dos funciones para mejor posibilidad de lectura, una que calcula el IMC y otra que lo clasifica.

- Exercici 2

Convertidor de temperatures.

Existeixen diverses unitats de temperatura utilitzades en diferents contextos i regions. Les més comunes són Celsius (°C), Fahrenheit (°F) i Kelvin (K). També existeixen altres unitats com Rankine (°Ra) i Réaumur (°Re). Selecciona almenys 2 conversors, de tal manera que en introduir una temperatura retorni, com a mínim, dues conversions.

```
tipo = input('¿Qué tipo de temperatura te gustaria convertir? a) Celsius b) Fahrenheit c) Kelvin: ')

if tipo in ['a', 'b', 'c']:
    try:
        temperatura = input('Dime la temperatura que te gustaria convertir: ')
        temperatura = temperatura.replace(',', '.')
        temperatura = float(temperatura)

        diccionario_temp = {
            'cf': round((temperatura * 9/5) + 32, 3),
            'ck': round(temperatura + 273.15, 3),
            'fc': round((temperatura - 32) * 5/9, 3),
            'fk': round((temperatura - 32) * 5/9 + 273.15, 3),
            'kc': round(temperatura - 273.15, 3),
            'kf': round((temperatura - 273.15) * 9/5 + 32, 3),
        }

        if tipo == 'a':
            print(f"{temperatura}° Celsius son: {diccionario_temp['cf']}° Fahrenheit y {diccionario_temp['ck']} Kelvin")
        elif tipo == 'b':
            print(f"{temperatura}° Fahrenheit son: {diccionario_temp['fc']}° Celsius y {diccionario_temp['fk']} Kelvin")
        elif tipo == 'c':
            print(f"{temperatura} Kelvin son: {diccionario_temp['kc']}° Celsius y {diccionario_temp['kf']}° Fahrenheit")

    except ValueError:
        print("Por favor, introduce un número válido para la temperatura.")
else:
    print("Opción no válida. Por favor selecciona a, b o c.")

conversor_de_temperaturas()
✓ 4.8s
25.0° Celsius son: 77° Fahrenheit y 298.15 Kelvin
```

Bueno, había empezado (y hecho) este ejercicio sin hacerlo por diccionario, o sea, apenas había creado `if` y `elif` para cada fórmula. Pero después, hablando con colegas y estudiando un poco más, me di cuenta de que la manera más fácil de leer el código era usando un diccionario. De hecho, si en el futuro quieres agregar una medida nueva, todo se vuelve un poco más simple. Pero una de las maneras más interesante sería crear un data frame con columnas, líneas y index y hacer una base de cálculo a partir de ahí, pero aun no lo sé hacerlo bien.

por eso, primero cree los tres tipo de medida y pedí al usuario cual es la medida que quieres inserir para ser convertida:

```
a) Celsius b) Fahrenheit c) Kelvin: ')
```

después de esto, y a partir de esto empecé pedindo la temperatura para el usuario y agregando la posibilidad de reemplazar (,) por (.) y de transformar el numero este en float.

creé el diccionario con las formulas de conversión de temperatura cf - Celsius a Fahrenheit, ck - Celsius a Kelvin y ETC.

Por fin, he creado esto:

```
if tipo == 'a':  
    print(f"{temperatura}° Celsius son:  
{diccionario_temp['cf']}° Fahrenheit y {diccionario_temp['ck']}  
Kelvin")  
    elif tipo == 'b':  
        print(f"{temperatura}° Fahrenheit son:  
{diccionario_temp['fc']}° Celsius y {diccionario_temp['fk']} Kelvin")  
        elif tipo == 'c':  
            print(f"{temperatura} Kelvin son:  
{diccionario_temp['kc']}° Celsius y {diccionario_temp['kf']}°  
Fahrenheit")
```

es decir: si el usuario selecciona a, b o c tendrá diferentes respuestas y para cada una tendrá dos tipos de conversiones.

- Exercici 3

Comptador de paraules d'un text.

Escriu una funció que donat un text, mostri les vegades que apareix cada paraula.

```
import re

def remove_pontos(lista):
    return re.sub(r'^\w\s', '', lista.lower())

def contador_de_palabras():

    formato_de_texto = input('Cual es el formato de tu texto? [a] - Quiero Digitarlo [b] formato (.txt): ')

    if formato_de_texto in ['a', 'b']:
        try:
            if formato_de_texto == 'a':
                texto = input('Digite tu texto: ')
                print(f'El texto digitado es: {texto}')
            elif formato_de_texto == 'b':
                camino = input('Digite el camino de el archivo: ')
                with open(camino, 'r', encoding='utf-8') as contenido:
                    texto = contenido.read()

            except Exception as e:
                print(f'Ocurrió un error: {e}')
        else:
            print("Opción no válida.")

    lista = texto.split()

    texto_ordenado = []

    for lineas in lista:
        palabra = remove_pontos(lineas).strip().lower().split()
        texto_ordenado.extend(palabra)
```



```
contagem_do_texto = {}

for palavras in texto_ordenado:
    if palavras in contagem_do_texto:
        contagem_do_texto[palavras] += 1
    else:
        contagem_do_texto[palavras] = 1

print('\nLa contagem de palabras és esta:')
return contagem_do_texto

contador_de_palabras()
```

[103]

```
... El texto digitado es: lucan ha ha ha skfj ajshd ajhsdsad

La contagem de palabras és esta:

... {'lucan': 1, 'ha': 3, 'skfj': 1, 'ajshd': 1, 'ajhsdsad': 1}
```

Este ejercicio pasó por muchas transformaciones. La primera versión, y más sencilla, fue crear un programa que hacía la cuenta de un texto que estaba insertado en una lista. Sin preocuparme por la creación de un DEF o de que este programa me funcionaria más tarde

en otros ejercicios.

Pero bueno, cuando llegué al ejercicio 1 del nivel 3 me di cuenta de que tendría que rehacer todo el SPRINT 7 pensando en un conjunto de códigos que pueden, de vez en cuando, ser utilizados en otros códigos. y fue esto que pasó con este ejercicio.

Después deshacer el último ejercicio del SPRINT 7 resolví crear un programa dentro de las necesidades de los ejercicios, por ejemplo:

```
formato_de_texto = input('Cual es el formato de tu texto? [a] - Quiero  
Digitarlo [b] formato (.txt): ')

if formato_de_texto in ['a', 'b']:

    try:

        if formato_de_texto == 'a':

            texto = input('Digite tu texto: ')

            print(f'El texto digitado es: {texto}')

        elif formato_de_texto == 'b':

            camino = input('Digite el camino de el archivo: ')

            with open(camino, 'r', encoding='utf-8') as contenido:

                texto = contenido.read()
```

creé este código para satisfacer dos necesidades:

- 1) si el usuario necesita hacer la cuenta de palabras de un texto digitado el tiene la opción A
- 2) si el usuario necesita hacer la cuenta de palabras de un texto en formato .TXT el tiene la opción B

después hice la limpieza del posible texto, quitando espacios, mayúsculas y puntuaciones para que mi futuro programa pudiera hacer la contagem de las palabras;
por fim cree un diccionario vacío donde se iba almacenar como clave el nombre de palabra y valor como el número de repeticiones.

- Exercici 4

Diccionari invers.

Resulta que el client té una enquesta molt antiga que s'emmagatzema en un diccionari i els resultats els necessita al revés, és a dir, intercanviats les claus i els valors. Els valors i claus en el diccionari original són únics; si aquest no és el cas, la funció hauria d'imprimir un missatge d'advertiment.

```
reverse_dictionary({'a': 1, 'b': 2, 'c': 3})
```

```
{1: 'a', 2: 'b', 3: 'c'}
```

```
reverse_dictionary({'x': 'apple', 'y': 'banana', 'z': 'banana'})
```

Error: multiple keys for one value

```
reverse_dictionary = {'x': 'apple', 'y': 'banana', 'z': 'banana'}

def reves_dic_error():
    print(reverse_dictionary)

    diccionario_inv = {}
    for key, value in reverse_dictionary.items():
        if value in diccionario_inv:
            print(f"\n!!!ERROR!!! TENEIS EL VALOR: '{value}' DUPLICADO.")
        else:
            diccionario_inv[value] = key

    print("\nEL DICIONARIO INVERTIDO ES:")

    return diccionario_inv
reves_dic_error()

{'x': 'apple', 'y': 'banana', 'z': 'banana'}

!!!ERROR!!! TENEIS EL VALOR: 'banana' DUPLICADO.

EL DICIONARIO INVERTIDO ES:

{'apple': 'x', 'banana': 'y'}
```

Ejercicio muy sencillo, cree un bucle con Clave y valor dentro de un diccionario vacío donde hay dos condiciones:

si hay un valor repetido escribir un mensaje

```
if value in diccionario_inv:
    print(f"\n!!!ERROR!!! TENEIS EL VALOR: '{value}'
DUPLICADO.")
```

y si no hay solo hacer la inversión de la clave por el valor. el problema es que como hay duplicado se elimina uno de los valores, arreglamos esto en el próximo ejercicio.

Nivell 2

Exercici 1

Diccionari invers amb duplicats

Continuant amb l'exercici 4 del nivell 1: al client es va oblidar de comentar un detall i resulta que els valors en el diccionari original poden duplicar-se i més, per la qual cosa les claus intercanviades poden tenir duplicats. En aquest cas, en l'exercici anterior imprimies un missatge d'avertiment, ara, els valors del diccionari resultant hauran d'emmagatzemar-se com una llista. Tingues en compte que si és un valor únic no ha de ser una llista.

```
reverse_dictionary_plus({'x': 'apple', 'y': 'banana', 'z': 'banana'})
```

```
{'apple': 'x', 'banana': ['y', 'z']}
```

```
def reves_dic_lista():  
    print("\nDICcionario ANTIGUO:")  
    print(reverse_dictionary)  
  
    diccionario_inv = {}  
    for key, value in reverse_dictionary.items():  
        if value in diccionario_inv:  
            if type(diccionario_inv[value]) != list:  
                diccionario_inv[value] = [diccionario_inv[value]]  
                diccionario_inv[value].append(key)  
            else:  
                diccionario_inv[value] = key  
        else:  
            diccionario_inv[value] = key  
  
    print("\nEL DICcionario INVERTIDO ES:")  
    return diccionario_inv  
  
reves_dic_lista()
```

```
DICcionario ANTIGUO:  
{'x': 'apple', 'y': 'banana', 'z': 'banana'}  
  
EL DICcionario INVERTIDO ES:  
{'apple': 'x', 'banana': ['y', 'z']}
```

es prácticamente el mismo ejercicio que el anterior, pero al contrario de producir un error, en este ejercicio creamos un loop en que los valores iguales son guardados como lista:

```
if type(diccionario_inv[value]) != list:
    diccionario_inv[value] = [diccionario_inv[value]]
    diccionario_inv[value].append(key)
```

o sea, estamos diciendo si el *value* (valor) del diccionario es = a otro *value*(valor), entonces insertamos los elementos como lista. Debemos insertar el valor como lista porque la lista acepta valores repetidos

Exercici 2

Conversió de tipus de dades

El client rep una llista de dades i necessita generar dues llistes, la primera on estaran tots els elements que es van poder convertir en flotants i l'altra on estan els elements que no es van poder convertir. Exemple de la llista que rep el client: ['1.3', 'one', '1e10', 'seven', '3-1/2', ('2', 1, 1.4, 'not-a-number'), [1, 2, '3', '3.4']]

```
conversion([ '1.3', 'one' , '1e10' , 'seven', '3-1/2', ('2',1,1.4,'not-a-number'), [1,2,'3','3.4']])
```

```
([1.3, 10000000000.0, 2.0, 1.0, 1.4, 1.0, 2.0, 3.0, 3.4],  
 ['one', 'seven', '3-1/2', 'not-a-number'])
```

```
lista_floats = ['1.3', 'one', '1e10', 'seven', '3-1/2', ('2', 1, 1.4, 'not-a-number'), [1, 2, '3', '3.4']]

def conversor_float_no_float():
    float_ = []
    n_float = []

    def conversor_float(value):
        try:
            return float(value)
        except (ValueError, TypeError):
            return None

    for item in lista_floats:
        if isinstance(item, (list, tuple)):
            for subitem in item:
                convert_value = conversor_float(subitem)
                if convert_value is not None:
                    float_.append(convert_value)
                else:
                    n_float.append(subitem)
            else:
                convert_value = conversor_float(item)
                if convert_value is not None:
                    float_.append(convert_value)
                else:
                    n_float.append(item)

    return {"floats": float_, "nofloats": n_float}

conversor_float_no_float()
```

✓ 0.0s

```
{'floats': [1.3, 10000000000.0, 2.0, 1.0, 1.4, 1.0, 2.0, 3.0, 3.4],  
 'nofloats': ['one', 'seven', '3-1/2', 'not-a-number']}
```

Para mí, este es el ejercicio más complejo del SPRINT 7, porque se trata de la creación de un código con la mayor cantidad de conceptos, y porque la visualización general es más compleja. Es decir, hay que tener un dominio más amplio de los conceptos de Python.

primero cree dos listas que es donde se van almacenar los floats y no floats

```
float_ = []
```

```
n_float = []
```

después cree un def que es un conversor de float

```
def conversor_float(value):  
    try:  
        return float(value)  
    except (ValueError, TypeError):  
        return None
```

Pero este conversor no sirve para todo, hay números que no pueden ser convertidos entonces hizo falta crear dos condiciones distintas.

```
for item in lista_floats:  
    if isinstance(item, (list, tuple)):  
        for subitem in item:  
            convert_value = conversor_float(subitem)  
            if convert_value is not None:  
                float_.append(convert_value)  
            else:  
                n_float.append(subitem)
```

En este caso, el bucle recorrerá elemento por elemento (usando `isinstance`, o sea con una función que busque objetos que son listas o tuplas) y, si el ítem es un `float`, lo agrega a la lista `float_[]`. Si el elemento está dentro de una lista o tupla, el bucle deberá recorrer sus sub ítems (elementos dentro de la lista o tupla).

En cada subítem, la regla será la misma: recorrer sus elementos, intentar convertirlos a `float`. Si se logra la conversión, el elemento se añadirá a la lista `FLOAT_`, y si no es posible, se agregará a la lista `N_FLOAT`.

El `else` se aplicará a los elementos fuera de las listas o tuplas, es decir, recorrerá esos elementos, intentará convertirlos en `float` y, si no se puede, los agrega a la lista `N_FLOAT`.

```
['1.3', 'one', '1e10', 'seven', '3-1/2', ('2', 1, 1.4, 'not-a-number'),  
[1, 2, '3', '3.4']]
```

Tenemos aquí una lista en la que el programa leerá como si hubiera 7 elementos, porque dentro de la lista principal hay una tupla.

```
('2', 1, 1.4, 'not-a-number')
```

y una otra lista

```
[1, 2, '3', '3.4']
```

por esto es necesario crear un bucle que pueda leer los los ítems y los ítems dentro de los ítems.

Nivell 3

Exercici 1

Comptador i endreçador de paraules d'un text.

El client va quedar content amb el comptador de paraules, però ara vol llegir arxius TXT i que calculi la freqüència de cada paraula ordenades dins de les entrades habituals del diccionari segons la lletra amb la qual comencen, és a dir, les claus han d'anar de la A a la Z i dins de la A hem d'anar de la A la Z. Per exemple, per a l'arxiu "tu_me_quieres_blanca.txt" la sortida esperada seria:

```
def diccionario():
    try:
        diccionario_de_palabras_desordenado = contador_de_palabras()

        diccionario_de_palabras = sorted(diccionario_de_palabras_desordenado.items(), key=lambda x: x[0])

        diccionario_alfabetico = {}

        for palabra, contagem in diccionario_de_palabras:
            primeiro_caractere = palabra[0].lower()
            if primeiro_caractere not in diccionario_alfabetico:
                diccionario_alfabetico[primeiro_caractere] = {}
            diccionario_alfabetico[primeiro_caractere][palavra] = contagem

        return diccionario_alfabetico
    except:
        print ('Por favor, hablar con el equipo de programacion')

diccionario()

La contagem de palabras és esta:

{'a': {'a': 3,
      'agua': 1,
      'al': 2,
      'alba': 4,
      'alcobas': 1,
      'alimenta': 1,
      'alma': 1,
      'amarga': 1,
      'azucena': 1},
 'b': {'baco': 1,
      'banquete': 1,
      'bebe': 1,
      'blanca': 3,
      'boca': 1,
      'bosques': 1,
```

Para este ejercicio utilicé el ejercicio 3 del nivel 1;

```
diccionario_de_palabras_desordenado = contador_de_palabras()
```

como ya he dicho en el ejercicio 3, este último nivel fue determinante para crear un raciocinio de como funciones pueden ser usadas dentro de otras funciones y que tu código funcione de manera más legible y orgánica;

la ultima cosa que he escrito en el código fue:

```
diccionario_de_palabras =
sorted(diccionario_de_palabras_desordenado.items(), key=lambda x: x[0])
```

porque el diccionario no me veía ordenado alfabéticamente, de hecho yo no había ordenado de esta manera en el ejercicio 3, esta manera de ordenar fue la más sencilla que encontré porque con el `sorted()` la ordené a partir de mi clave.

```
for palavra, contagem in dicionario_de_palavras:
    primeiro_caractere = palavra[0].lower()
    if primeiro_caractere not in dicionario_alfabetico:
        dicionario_alfabetico[primeiro_caractere] = {}
    dicionario_alfabetico[primeiro_caractere][palavra] =
contagem
```

De hecho este Bucle que cree sirve para crear un diccionario a partir de las primeras letras de un texto y después agregar dentro de este diccionario de letras el conteo del contador de palabras.

el `not in` hace con que se corra letras que no son repetidas,